

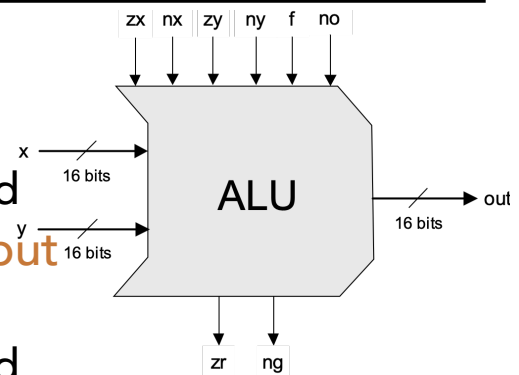
With a little help from my friends - implementing ALU

- Managing inputs
 - create logic circuits which, on the basis of **zx**, **nx**, **zy**, **ny** are able to **select** the proper 16-bit inputs
- Managing operators/outputs
 - create logic circuits which, on the basis of **f**, **no** are able to **select** the proper operation to be emitted
- Main idea
 - everything is computed altogether - the problem is **selecting** (i.e. MUXing)

25

With a little help from my friends - implementing ALU

- As concerning the **first input**
 - it can be zero or x
 - it can be plain or bit-inverted
- As concerning the **second input**
 - it can be zero or y
 - it can be plain or bit-inverted
- As concerning **operations and output**
 - both ADD and AND must be evaluated
 - final output can be plain or bit-inverted
- As concerning the **status bits**
 - they codify some kind of semantic meanings

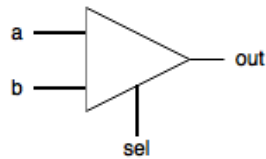


26

With a little help from my friends - implementing ALU

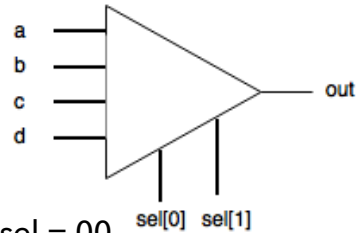
■ Selecting with MUX

Selecting between 2 alternatives



IF sel = 0
out = a
ELSE
out = b

Selecting among 4 alternatives

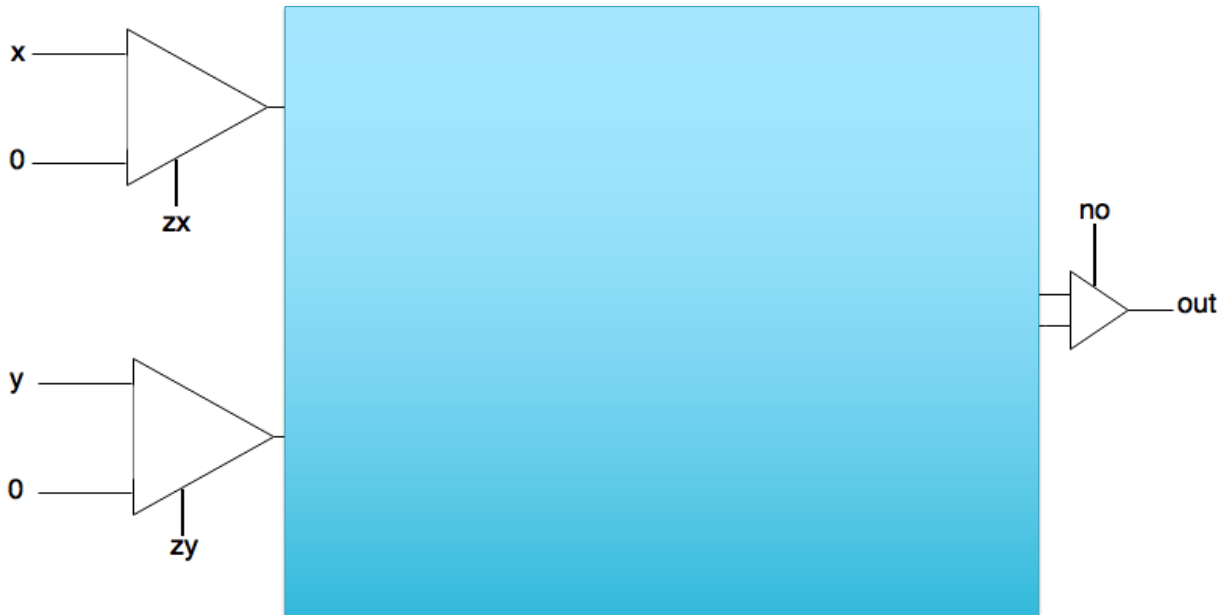


IF sel = 00
out = a
ELSEIF sel = 01
out = b
...
ELSEIF sel = 11
out = d

27

With a little help from my friends - implementing ALU

■ ALU implementation diagram 2



30

With a little help from my friends - tips for project 2

- For half adder two known gates can be used
- Full adder can be made up of 2 half adder (or in some other way)
- An n-bit adder can be built from n full adder chips
 - the carry bits are moved from right to left
 - the most significative-related carry is ignored
- For Inc16 remember that single "0", "1" bits can be expressed as "false", "true" in HDL
- ALU can be built with less than 20 parts
- Respect the given order and use built-in project-1 chips

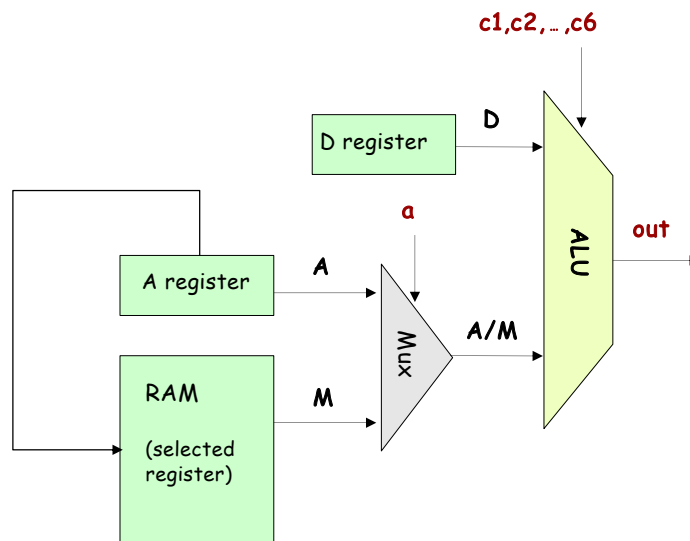
31

With a little help from my friends - delivery instructions

- Please adhere to the following notation for delivering the .hdl files
 - submit a zip file containing **all (and only)** the 5 .hdl files
 - pay **attention to respect** the following syntax for the name of the zip file
_P02CognomeNome.zip (containing immediately 5 hdl files, with **no additional directory**)

33

The ALU in the CPU context (a sneak preview of the Hack platform)



34

The ALU in action inside the hardware simulator

The screenshot shows the hardware simulator interface. The top menu bar includes File, View, Run, and Help. The toolbar contains various icons for simulation control. The main window is divided into several sections:

- 2. Evaluate the chip**: A callout box pointing to the 'Evaluate' button in the toolbar.
- 1. Set the ALU inputs and control bits to some test values (000111) means "y-x"**: A callout box pointing to the 'ALU' block in the HDL section.
- Built in ALU implementation**: A callout box pointing to the 'ALU' block in the HDL section.

The HDL section displays the following code:

```
// This file is part of www.nanc
// and the book "The Elements of
// by Nisan and Schocken, MIT Pr
// File name: tools/builtIn/ALU.

/**
 * The ALU. Computes one of the
 * * x+y, x-y, y0x, 0, 1, -1, x, y
 * * x+1, y+1, x-1, y-1, x&y, x|y
 * * Which function to compute i
 * * denoted zx, nx, zy, ny, f, nc
 * * The computed function's value
 * * In addition to computing out,
 * * 1-bit outputs called zr and r
```

The 'Output pins' table shows the following values:

Name	Value
out[16]	0
zr	1
ng	0

The 'ALU' block in the HDL section shows the following inputs and output:

- D Input: 0
- M/A Input: 0
- ALU output: 0

36

The ALU in action inside the hardware simulator

2. Evaluate the chip

3. Inspect the ALU output

1. Set the ALU inputs and control bits to some test values (000111) means "y-x"

Built in ALU implementation

The built-in ALU implementation has some GUI side-effects

File View Run Help

Time : 0

Output pins

Name	Value
out[16]	30
zr	0
ng	1

HDL

```
// This file is part of www.nanc
// and the book "The Elements of
// by Nisan and Schocken, MIT Pr
// File name: tools/builtIn/ALU.

/**
 * The ALU. Computes one of the
 * x+y, x-y, y&x, 0, 1, -1, x, y
 * x+1, y+1, x-1, y-1, x&y, x|y
 * Which function to compute is
 * denoted zx, nx, zy, ny, f, nc
 * The computed function's value
 * In addition to computing out,
 * 1-bit outputs called zr and r
```

37

Set to binary I/O format

Inspect the ALU output

Set the ALU inputs and control bits to some test values (000000) means "x&y"

File View Run Help

Time : 0

Input pins

Name	Value
x[16]	1110101110000110
y[16]	000110000110101
zx	0
nx	0
zy	0
ny	0
f	0
no	0

Output pins

Name	Value
out[16]	000010000000100
zr	0
ng	0

HDL

```
// This file is part of www.nanc
// and the book "The Elements of
// by Nisan and Schocken, MIT Pr
// File name: tools/builtIn/ALU.

/**
 * The ALU. Computes one of the
 * x+y, x-y, y&x, 0, 1, -1, x, y
 * x+1, y+1, x-1, y-1, x&y, x|y
 * Which function to compute is
 * denoted zx, nx, zy, ny, f, nc
 * The computed function's value
 * In addition to computing out,
 * 1-bit outputs called zr and r
```

39

Perspective

- Combinational logic
- Our ALU is also very basic: no multiplication, no division
- Where is the seat of more advanced math operations?
a typical hardware/software tradeoff.

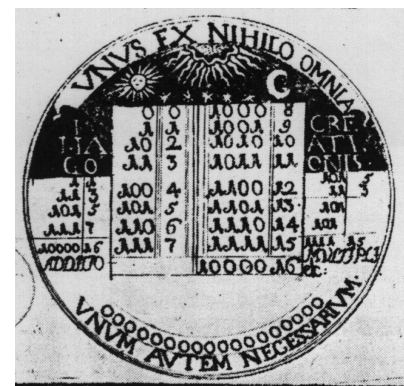
41

Historical end-note: Leibnitz (1646-1716)

- "The binary system may be used in place of the decimal system; express all numbers by unity and by nothing"
- 1679: built a mechanical calculator (+, -, *, /)



- CHALLENGE: "All who are occupied with the reading or writing of scientific literature have assuredly very often felt the want of a common scientific language, and regretted the great loss of time and trouble caused by the multiplicity of languages employed in scientific literature:
- SOLUTION: "Characteristica Universalis": a universal, formal, and decidable language of reasoning
- The dream's end: Turing and Gödel in 1930's.



Leibniz's medallion
for the Duke of Brunswick

42